INF201 – Deliverable 2

Task 1: File Ingestion and Regular Expressions

Task 1.1 - File Ingestion

In [6]:
```
from pathlib import Path
import re
```

In [7]:
```
def fileread(filepath: str | Path) -> tuple[str, str]:
    """
    Read a text file using UTF-8 or Latin-1 encoding.

    Args:
        filepath (str | Path): Path to the file.

    Returns:
        tuple[str, str]: File content and the encoding used.
    """

    path = Path(filepath)

    # Try reading the file with UTF-8 encoding first
    try:
        with open(path, "r", encoding="utf-8") as file:
            content = file.read()
            encoding_used = "utf-8"

    # If UTF-8 fails, try Latin-1 encoding instead
    except UnicodeDecodeError:
        with open(path, "r", encoding="latin-1") as file:
            content = file.read()
            encoding_used = "latin-1"

    # Return both the file content and the encoding used
    return content, encoding_used
```

In [8]:
```
content, encoding_used = fileread(
    Path("data/2026/science_building/POS_01/register_sales.log")
)

print(content, encoding_used)
```

```
[2026-10-15 07:00:49] [POS_01] item=Croissant price=4.14 EUR (REFUND)
[2026-10-15 07:02:05] [POS_01] item=Cappuccino price=4.50 NOK (SUCCESS)
[DEBUG] BARCODE SCANNER REBOOTED
[2026-10-15 07:02:26] [POS_01] item=Espresso price=2.76 USD (SUCCESS)
[2026-10-15 07:03:40] [POS_01] item=Croissant price=999.00 EUR (CANCELLED)
[2026-10-15 07:06:01] [POS_01] item=Muffin price=3.06 NOK (SUCCESS)
[2026-10-15 07:06:41] [POS_01] item=Croissant price=3.76 USD (REFUND)
[2026-10-15 07:09:10] [POS_01] item=Muffin price=3.00 NOK (REFUND)
[2026-10-15 07:11:04] [POS_01] item=Espresso price=2.59 NOK (SUCCESS)
[2026-10-15 07:12:51] [POS_01] item=Croissant price=4.10 NOK (SUCCESS)
[2026-10-15 07:15:01] [POS_01] item=Espresso price=2.96 NOK (SUCCESS)
[2026-10-15 07:15:12] [POS_01] item=Croissant price=3.70 NOK (REFUND)
[2026-10-15 07:16:56] [POS_01] item=Sandwich price=7.64 NOK (REFUND)
[2026-10-15 07:18:12] [POS_01] item=Cappuccino price=4.31 USD (SUCCESS)
[2026-10-15 07:20:28] [POS_01] item=Latte price=4.33 NOK (SUCCESS)
[DEBUG] BARCODE SCANNER REBOOTED
[2026-10-15 07:21:35] [POS_01] item=Muffin price=3.42 NOK (SUCCESS)
[2026-10-15 07:23:33] [POS_01] item=Muffin price=3.12 EUR (SUCCESS)
=== NETWORK SYNC OK ===
[2026-10-15 07:24:55] [POS_01] item=Muffin price=3.82 NOK (SUCCESS)
[2026-10-15 07:26:32] [POS_01] item=Espresso price=2.85 NOK (SUCCESS)
[2026-10-15 07:27:53] [POS_01] item=Muffin price=3.82 EUR (SUCCESS)
=== NETWORK SYNC OK ===
[2026-10-15 07:30:09] [POS_01] item=Muffin price=999.00 NOK (REFUND)
[2026-10-15 07:32:15] [POS_01] item=Sandwich price=7.50 EUR (SUCCESS)
[2026-10-15 07:33:21] [POS_01] item=Sandwich price=7.46 EUR (SUCCESS)
[2026-10-15 07:36:01] [POS_01] item=Muffin price=3.92 USD (SUCCESS)
--- PRINTER PAPER LOW ---
[2026-10-15 07:38:09] [POS_01] item=Cappuccino price=4.38 NOK (REFUND)
[DEBUG] BARCODE SCANNER REBOOTED
[2026-10-15 07:39:35] [POS_01] item=Muffin price=3.83 EUR (CANCELLED)
[2026-10-15 07:40:22] [POS_01] item=Latte price=4.85 NOK (SUCCESS)
[2026-10-15 07:41:43] [POS_01] item=Latte price=4.85 EUR (SUCCESS)
[2026-10-15 07:42:16] [POS_01] item=Latte price=4.92 USD (CANCELLED)
[2026-10-15 07:44:13] [POS_01] item=Espresso price=2.71 USD (SUCCESS)
[2026-10-15 07:45:02] [POS_01] item=Cappuccino price=4.15 NOK (CANCELLED)
[2026-10-15 07:47:13] [POS_01] item=Cappuccino price=4.31 NOK (SUCCESS)
[2026-10-15 07:49:59] [POS_01] item=Croissant price=3.33 USD (SUCCESS)
[2026-10-15 07:52:31] [POS_01] item=Sandwich price=7.00 EUR (SUCCESS)
[2026-10-15 07:54:25] [POS_01] item=Croissant price=4.00 EUR (REFUND)
[2026-10-15 07:55:13] [POS_01] item=Espresso price=3.35 NOK (CANCELLED)
[2026-10-15 07:58:10] [POS_01] item=Muffin price=999.00 NOK (REFUND)
[2026-10-15 07:58:58] [POS_01] item=Cappuccino price=3.92 NOK (SUCCESS)
[2026-10-15 08:01:05] [POS_01] item=Sandwich price=7.57 EUR (CANCELLED)
[2026-10-15 08:03:48] [POS_01] item=Latte price=4.95 USD (SUCCESS)
--- PRINTER PAPER LOW ---
[2026-10-15 08:04:43] [POS_01] item=Latte price=4.50 USD (SUCCESS)
=== NETWORK SYNC OK ===
[2026-10-15 08:05:05] [POS_01] item=Croissant price=4.17 EUR (SUCCESS)
[2026-10-15 08:06:10] [POS_01] item=Latte price=4.26 EUR (SUCCESS)
[2026-10-15 08:09:09] [POS_01] item=Croissant price=4.29 NOK (SUCCESS)
[2026-10-15 08:11:07] [POS_01] item=Latte price=4.94 USD (SUCCESS)
[2026-10-15 08:12:34] [POS_01] item=Cappuccino price=4.59 NOK (SUCCESS)
[2026-10-15 08:13:14] [POS_01] item=Muffin price=3.94 EUR (SUCCESS)
[2026-10-15 08:15:34] [POS_01] item=Cappuccino price=4.26 EUR (SUCCESS)
[2026-10-15 08:16:05] [POS_01] item=Muffin price=3.14 NOK (SUCCESS)
[2026-10-15 08:16:16] [POS_01] item=Croissant price=4.20 USD (SUCCESS)
[2026-10-15 08:17:59] [POS_01] item=Muffin price=3.88 EUR (SUCCESS)
[2026-10-15 08:18:45] [POS_01] item=Croissant price=4.28 EUR (SUCCESS)
[2026-10-15 08:21:32] [POS_01] item=Cappuccino price=4.22 USD (SUCCESS)
```

```
[2026-10-15 08:22:54] [POS_01] item=Croissant price=3.51 EUR (SUCCESS)
[2026-10-15 08:25:16] [POS_01] item=Espresso price=3.27 EUR (SUCCESS)
[2026-10-15 08:27:37] [POS_01] item=Muffin price=3.58 NOK (SUCCESS)
[2026-10-15 08:28:29] [POS_01] item=Muffin price=3.27 NOK (SUCCESS)
[2026-10-15 08:31:13] [POS_01] item=Muffin price=3.44 USD (SUCCESS)
[2026-10-15 08:33:55] [POS_01] item=Croissant price=3.33 NOK (SUCCESS)
[2026-10-15 08:34:54] [POS_01] item=Cappuccino price=4.66 NOK (SUCCESS)
[2026-10-15 08:36:01] [POS_01] item=Sandwich price=7.59 NOK (SUCCESS)
[2026-10-15 08:38:43] [POS_01] item=Sandwich price=7.56 USD (SUCCESS)
[2026-10-15 08:39:56] [POS_01] item=Sandwich price=7.10 USD (REFUND)
[2026-10-15 08:40:34] [POS_01] item=Muffin price=3.75 USD (SUCCESS)
[2026-10-15 08:41:18] [POS_01] item=Sandwich price=7.14 USD (SUCCESS)
[2026-10-15 08:41:44] [POS_01] item=Latte price=4.07 EUR (REFUND)
[2026-10-15 08:43:51] [POS_01] item=Muffin price=3.70 EUR (SUCCESS)
=== NETWORK SYNC OK ===
[2026-10-15 08:45:57] [POS_01] item=Espresso price=2.94 EUR (SUCCESS)
[2026-10-15 08:48:36] [POS_01] item=Croissant price=3.94 EUR (CANCELLED)
[2026-10-15 08:49:29] [POS_01] item=Croissant price=4.21 NOK (SUCCESS)
[2026-10-15 08:50:32] [POS_01] item=Muffin price=3.09 EUR (SUCCESS)
[2026-10-15 08:52:58] [POS_01] item=Croissant price=3.86 USD (SUCCESS)
[2026-10-15 08:54:25] [POS_01] item=Espresso price=2.60 EUR (SUCCESS)
[2026-10-15 08:56:11] [POS_01] item=Muffin price=3.84 EUR (SUCCESS)
[2026-10-15 08:56:57] [POS_01] item=Croissant price=3.89 NOK (SUCCESS)
[2026-10-15 08:58:41] [POS_01] item=Latte price=4.42 EUR (CANCELLED)
=== NETWORK SYNC OK ===
[2026-10-15 08:59:00] [POS_01] item=Sandwich price=7.18 EUR (SUCCESS)
[2026-10-15 09:01:24] [POS_01] item=Latte price=4.87 USD (SUCCESS)
[2026-10-15 09:02:20] [POS_01] item=Cappuccino price=3.79 EUR (SUCCESS)
[2026-10-15 09:03:11] [POS_01] item=Croissant price=4.00 EUR (REFUND)
[2026-10-15 09:05:02] [POS_01] item=Cappuccino price=4.53 EUR (SUCCESS)
 utf-8
```

Task 1.1 – Reading files

I created a function that reads text files using UTF-8 encoding, with Latin-1 as a fallback if UTF-8 fails. The function uses `try` and `except UnicodeDecodeError` to handle different encodings and returns both the file content and the encoding used. I tested the function on a generated log file and confirmed that it successfully read the content.

Task 1.2 - Regex and transaction management

In [9]:

```python
def parse_transactions(content: str) -> list[dict]:
    """
    Extract transactions from log file content using regex.

    Args:
        content (str): Text content from a log file.

    Returns:
        list[dict]: List of dictionaries containing transaction data.
    """

    # Create an empty list to store the transactions
    transactions = []

    # Go through the log file one line at a time
    for line in content.splitlines():
```

```
        # Search for transaction data using six capture groups
        match = re.search(
            r"\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s\[(\w+)\]\sitem=(\w+)\s
            line
        )

        # Only process lines that match the transaction pattern
        if match:

            # Store the extracted values in a dictionary
            values = {
                "timestamp": match.group(1),
                "register_id": match.group(2),
                "item": match.group(3),
                "price": float(match.group(4)),
                "currency": match.group(5),
                "status": match.group(6)
            }

            # Add the transaction dictionary to the list
            transactions.append(values)

    # Return all extracted transactions
    return transactions
```

In [10]:
```
transactions = parse_transactions(content)
transactions[:3]
```

Out[10]:
```
[{'timestamp': '2026-10-15 07:00:49',
  'register_id': 'POS_01',
  'item': 'Croissant',
  'price': 4.14,
  'currency': 'EUR',
  'status': 'REFUND'},
 {'timestamp': '2026-10-15 07:02:05',
  'register_id': 'POS_01',
  'item': 'Cappuccino',
  'price': 4.5,
  'currency': 'NOK',
  'status': 'SUCCESS'},
 {'timestamp': '2026-10-15 07:02:26',
  'register_id': 'POS_01',
  'item': 'Espresso',
  'price': 2.76,
  'currency': 'USD',
  'status': 'SUCCESS'}]
```

Task 1.2 – Parsing transactions

I created a function that uses regular expressions to extract transaction data from the log file. Each transaction is stored as a dictionary containing the timestamp, register ID, item, price, currency and status, with the price converted to a float. Testing showed that the function successfully extracted 80 transactions from 90 lines, correctly ignoring the 10 system messages.